COP-4534
Algorithm Techniques

Instructor: Dr. Antonio Hernandez

Text: The Algorithm Design Manual, 3rd Edition

Algorithmic Techniques

19. Combinatorial Optimization

Combinatorial Optimization

Combinatorial Optimization

Finding an object of the solution space pertaining to a combinatorial problem, that maximizes or minimizes a given objective function

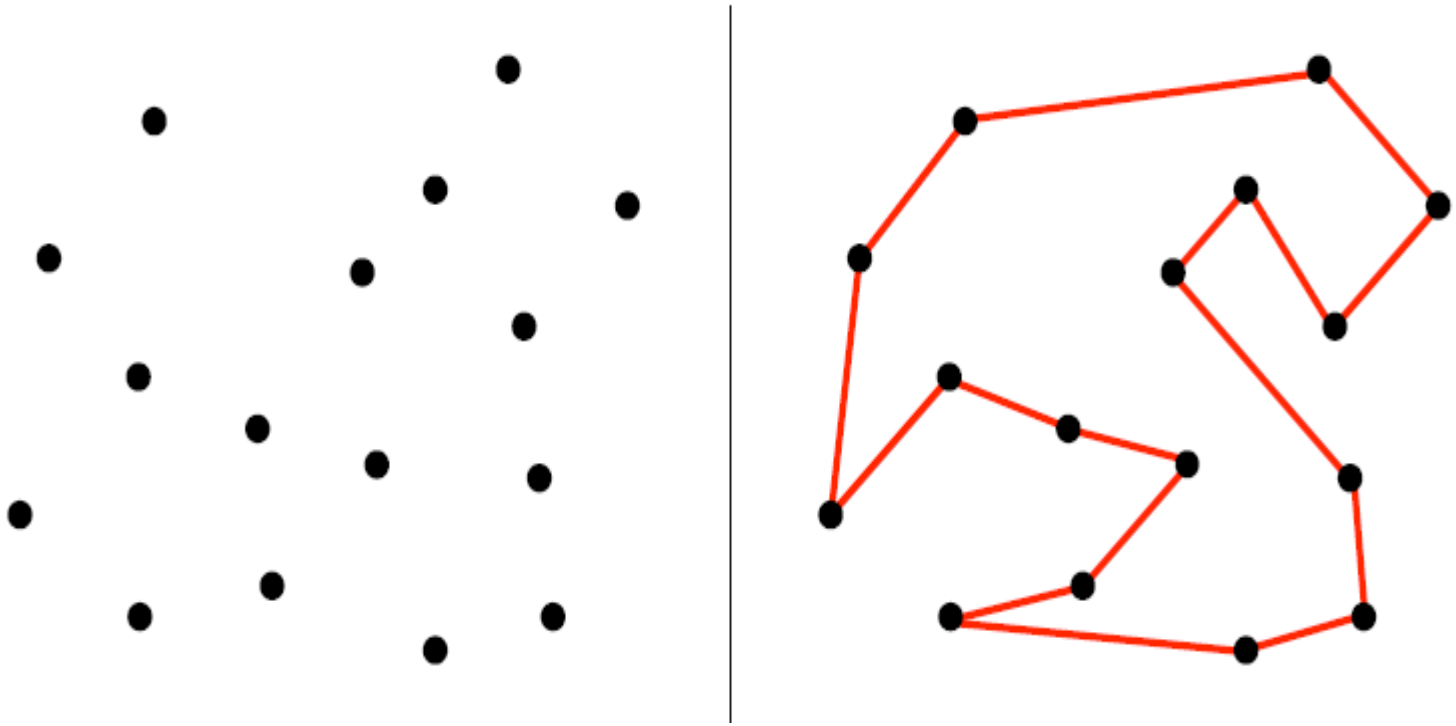
(*) C.O. problems are often hard problems!

Traveling Salesman Problem

Traveling Salesman Problem (TSP)

Given n cities and the pairwise distances, what is a shortest route that visits each city exactly once and returns to the starting city?

Traveling Salesman Problem



(*) Figure taken from class textbook

Traveling Salesman Problem

TSP as a Graph Problem

Given a complete weighted graph with n vertices, find a shortest route that visits every vertex exactly once and returns to the starting vertex.

(*) equivalently, in graph terms: a Hamiltonian cycle with minimum cost)

Combinatorial Optimization

Approaching Combinatorial Optimization

- Exhaustive search (brute-force)
- Heuristic method
- Approximation Algorithms

Combinatorial Optimization

Exhaustive search solution to TSP

- Find all permutations of the vertices of the graph
- Each permutation is a cycle: compute the cost of each
- Determine cycle with smallest cost

Generating All Permutations: **Review of Prog16_02**

```
/**
 * Print all permutations with values in [0, n-1].
 */
public void printPermutations(int n)
{
    int[] a = new int[n];
    printPermutations(a, 0);
}

/**
 * Recursive algorithm.
 *
 * @param a array partially filled with permutation
 * @param k index of current element in permutation
 */
private void printPermutations(int[] a, int k)
{
```

Generating All Permutations: **Review of Prog16_02**

```
private void printPermutations(int[] a, int k)
{
    if (k == a.length)
    {
        printArray(a);
    }
    else
    {
        ArrayList<Integer> Sk =
                                constructCandidateSet(a, k);
        for (int s : Sk)
        {
            a[k] = s;
            printPermutations(a, k+1);
        }
    }
}
```

Generating All Permutations: **Review of Prog16_02**

```java
/**
 * Construct candidate set (set will contain elements
 * not used in locations [0, k-1] of array a).
 */
private ArrayList<Integer>
                constructCandidateSet(int[] a, int k) {
    ArrayList<Integer> candidates = new ArrayList<>();
    boolean[] b = new boolean[a.length];

    for (int i=0; i<k; i++) {
        b[a[i]] = true;
    }

    for(int i=0; i<a.length; i++) {
        if (!b[i]) candidates.add(i);
    }

    return candidates;
}
```

Prog19_01: TSP solved with Brute-Force

```
/** Finds a shortest route that visits every vertex
    * exactly once and returns to the starting point.
    * Uses exhaustive search.
    *
    * @param shortestRoute shortest path (return value)
    *
    * @return shortest distance
    */
   public int TSP_exhaustiveSearch(int[] shortestRoute) {
       // initialize shortestRoute
       for (int i=0; i<verticesNumber; i++) {
           shortestRoute[i] = i;
       }

       int[] a = new int[verticesNumber];
       TSP_exhaustiveSearch(shortestRoute, a, 0);

       return totalDistance(shortestRoute);
   }
```

Prog19_01: TSP solved with Brute-Force

```
/** Calculates distance of given route.
 *
 * @param a route
 *
 * @return distance of route
 */
int totalDistance(int[] a) {
    int n = verticesNumber;

    //add weights of all edges in the path
    int totalWeight = 0;
    for(int i=0; i<n; i++) {
        int weight = matrix[a[i]][a[(i+1)%n]];
        totalWeight += weight;
    }

    return totalWeight;
}
```

Prog19_01: TSP solved with Brute-Force

```
/**
 * Recursive algorithm.
 *
 * @param a array partially filled with permutation
 * @param k index of current element in permutation
 */
 private void TSP_exhaustiveSearch(int[] shortestRoute,
                                   int[] a, int k) {
    if (k == a.length) {
       if (totalDistance(a) < totalDistance(shortestRoute)) {
              System.arraycopy(a, 0, shortestRoute, 0,
                               verticesNumber);
           }
           //System.out.print(totalDistance(a) + " ");
           //printArray(a);

       }
```

Prog19_01: TSP solved with Brute-Force

```
        else {
            ArrayList<Integer> Sk =
                                constructCandidateSet(a, k);
            for (int s : Sk) {
                a[k] = s;
                TSP_exhaustiveSearch(shortestRoute, a, k+1);
            }
        }
    }
```

Prog19_01: TSP solved with Brute-Force

```
/** Construct candidate set (set will contain elements
 * not used in locations [0, k-1] of array a).
 */
private ArrayList<Integer> constructCandidateSet(int[] a,
                                                 int k) {
    ArrayList<Integer> candidates = new ArrayList<>();
    boolean[] b = new boolean[a.length];

    for (int i=0; i<k; i++) {
        b[a[i]] = true;
    }

    for(int i=0; i<a.length; i++) {
        if (!b[i]) candidates.add(i);
    }

    return candidates;
}
```

PRACTICE

Program 19_01:

Add to the Graph class the code just discussed to solve TSP (given in Canvas) and test.

Heuristic Method: Random Sampling

Random Sampling

While stopping criterion is not met, generate and evaluate random candidate solutions and take the best one of all.

Random Sampling

Random Sampling

```
Y = initial solution
while (stopping criterion is not met)
{
   X = randomly generated candidate solution
   if (evaluation(X) is better than evaluation(Y))
        Y ← X
}
return Y
```

Random Sampling

Random Sampling

- Examples of stopping criterion: fixed length of time has elapsed, given number of iterations has been reached, certain desirable feature has been obtained
- Each element of the solution space must have an equal probability of being generated

Prog19_02: Random Permutations

```
import java.util.Arrays;
import java.util.Random;

public class Prog19_02 {
    public static void main(String[] args) {
        new Prog19_02();
    }

    public Prog19_02() {

        //generate 10 random permutations of values in [0, 5]
        for(int i=0; i<10; i++) {
            int[] a = new int[6];
            randomPermutation(a);

            System.out.println(Arrays.toString(a));
        }
    }
```

Prog19_02: Random Permutations

```
/** Generates random permutation of values in 0, n-1].
 * Uses Fisher-Yates shuffle algorithm. */
public void randomPermutation(int[] a) {
    for (int i=0; i<a.length; i++)
        a[i] = i;

    Random rnd = new Random();
    for (int i=a.length-1; i>0; i--) {
        //generates a random index in [0, i]
        int randomLocation = rnd.nextInt(i+1);

        if (randomLocation != i) {
            //swap a[i] and a[randomLocation]
            int temp = a[i];
            a[i] = a[randomLocation];
            a[randomLocation] = temp;
        }
    }
} }
```

PRACTICE

Program 19_02:

Test the code just discussed to generate random permutations (given in Canvas).

Prog19_03: TSP with Random Sampling

```
public int TSP_randomSampling(int[] shortestRoute) {

    int numberOfSamples = 10;
    int bestDistance = Integer.MAX_VALUE;

    for (int i=0; i< numberOfSamples; i++) {
        int[] a = new int[verticesNumber];
        randomPermutation(a);
        int currentDistance = totalDistance(a);
        if (currentDistance < bestDistance) {
            bestDistance = currentDistance;
            System.arraycopy(a, 0,
                             shortestRoute, 0,
                             verticesNumber);
        }
    }

    return bestDistance;
}
```

PRACTICE

Program 19_03:

Test the code just discussed to find a solution to TSP using random sampling (given in Canvas).

Random Sampling

When to Use Random Sampling?

- When there is a large proportion of acceptable solutions

  Example: generation of large prime numbers

- When there is no structure in the solution space

  Not common among combinatorial problems

Local Search

Local Search

Starting in an initial candidate solution, the method iteratively moves to a neighbor solution that improves the value of the objective function. It stops when no better solution is found in the neighborhood of the current candidate solution.

Local Search

Local Search

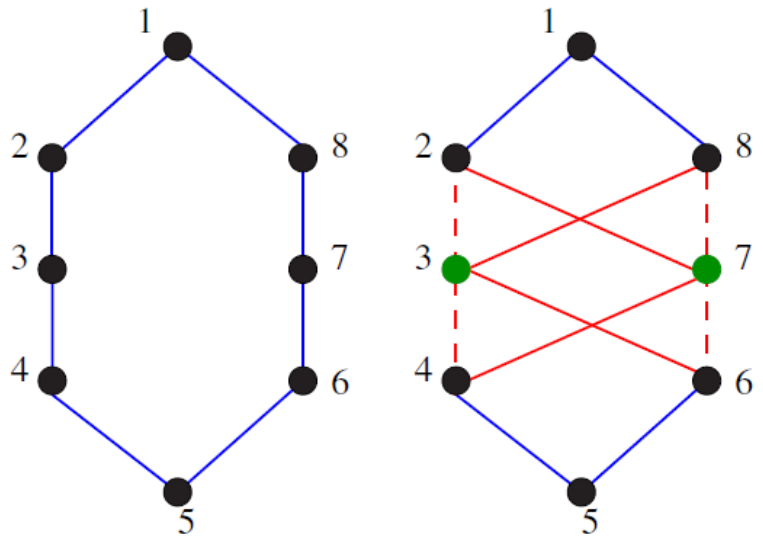
```
Y = initial candidate solution
do {
   N = neighborhood  of Y
   for each X in N {
      if (evaluation(X) is better than evaluation(Y))
         Y ← X
   }
} while (Y is found)

return Y
```

Local Search: TSP

Neighborhood in TSP

Swap two random vertices in the current best route.



Swap 3 and 7

(*) Figure taken from class textbook

The Iterator Design Pattern

The Iterator Design Pattern

Defines a way to access and traverse a group of elements. Defines methods **hasNext** and **next**.

boolean hasNext():

true iff at least an element remains to be accessed

ElemenType next(): returns next element

The Iterator Design Pattern

Iterator **it**: iterator of group of elements

```
while (it.hasNext())
{
    ElementType e = it.next();
    //process e
}
```

Prog19_04: All Neighbor Permutations

```
/** Iterator class, allows traversal of the set of neighbors
 * of given permutation. There will be n(n-1)/2 neighbors,
 * obtained by swapping all pairs of elements in the
 * permutation.
 */
public class PermutationNeighborhood {

    private final int[] p; //permutation
    private final int SIZE; //size of permutation
    private int loc1; //loc1 and loc2 are the locations of
    private int loc2; //p that will be swapped next

    /** Initializes permutation of this object and locations
     * whose values will be swapped.
     *
     * @param a permutation whose neighborhood is to
     * be generated
     */
    public PermutationNeighborhood(int[] a) {
```

Prog19_04: All Neighbor Permutations

```
    public PermutationNeighborhood(int[] a) {
        SIZE = a.length;
        p = new int[SIZE];
        System.arraycopy(a, 0, p, 0, SIZE);

        loc1 = 0;
        loc2 = 1;
    }

    /** Returns true if at least a neighbor remains to be
     * generated; false otherwise.
     */
    public boolean hasNext() {
        return loc1 != SIZE - 1;
    }

    /** Returns next permutation neighbor if one exists;
     * null otherwise */
    public int[] next() {
```

Prog19_04: All Neighbor Permutations

```
public int[] next() {
    if (hasNext()) {
        //copy p to a
        int[] a = new int[SIZE];
        System.arraycopy(p, 0, a, 0, SIZE);

        //exchange elements at locations loc1 and loc2
        a[loc1] = p[loc2];
        a[loc2] = p[loc1];

        //advance loc1 and loc2
        if (loc2 == SIZE - 1) {
            loc1++;
            loc2 = loc1 + 1;
        }
        else loc2++;

        return a;
    }
```

Prog19_04: All Neighbor Permutations

```
        else return null;
    }
}
```

PRACTICE

Program 19_04:

Test the code just discussed to generate the neighborhood of a given permutation (given in Canvas).

Prog19_05: TSP with Local Search

```
    public int TSP_localSearch(int[] shortestRoute) {
        int bestDistance;

        //generate initial solution as a random permutation
        int[] a = new int[verticesNumber];
        randomPermutation(a);

        //shortestRoute = initial solution
        System.arraycopy(a, 0,
                         shortestRoute, 0, verticesNumber);
        bestDistance = totalDistance(shortestRoute);

        boolean betterSolutionFound;

        //Iteration loop
        do
        {
```

Prog19_05: TSP with Local Search

```
        //Iteration loop
        do {
            betterSolutionFound = false;
            PermutationNeighborhood pn = new
                      PermutationNeighborhood(shortestRoute);
            while (pn.hasNext()) {
                a = pn.next();
                int currentDistance = totalDistance(a);
                if (currentDistance < bestDistance) {
                    //shortestRoute = current solution
                    System.arraycopy(a, 0,
                           shortestRoute, 0, verticesNumber);
                    bestDistance = currentDistance;
                    betterSolutionFound = true;
                }
            }
        } while (betterSolutionFound);

        return bestDistance;
    }
```

PRACTICE

Program 19_05:

Test the code just discussed to find a solution to TSP using local search (given in Canvas).